AI Security Findings

Overall risk: CRITICAL | Total findings: 15

Executive summary

Risk overview

Total findings: 15. Critical: 4, High: 9, Medium: 2, Low: 0.

Technical summary

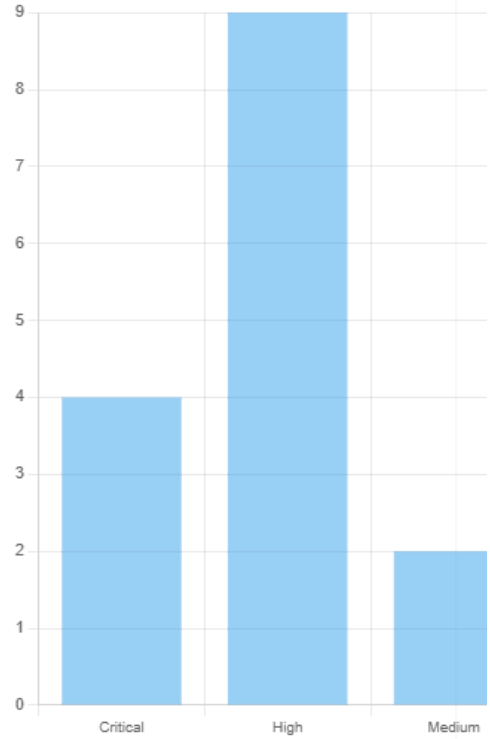
Overall Risk Level: Critical

Key Issues to Fix First:
1. Command Injection Vulnerability – The use of os.popen() with unsanitized user input in multiple endpoints (e.g., /run) must be replaced with safer alternatives like subprocess.run() with strict input validations or whitelist of allowed commands.
2. SQL Injection Vulnerabilities – Multiple functions build SQL queries with string concatenation using untrusted input. Switch to parameterized queries or prepared statements to prevent arbitrary SQL execution.
3. Hardcoded Secrets and Sensitive Credentials – Hardcoded API keys, tokens, and database credentials in source code and configuration files expose the system if the code leaks. Transition to environment variables or a secure secrets management system.
4. Debug Mode and Sensitive Information Exposure – Running the application in DEBUG mode and exposing configuration files via endpoints can reveal internal state and sensitive data. Disable debug mode for production and protect sensitive data exposure.

Quick Wins & Hardening Recommendations:
• Immediately disable DEBUG mode in production and audit access to configuration endpoints.
• Remove hardcoded credentials and secrets; enforce secure storage and access controls.
• Update all vulnerable code segments to use secure coding practices—validate user inputs thoroughly and log minimal sensitive information (e.g., avoid logging plain text passwords).
• Consider integrating a robust authentication framework for secure token verification and employing client-side safeguards to prevent XSS vulnerabilities.

Implementing these fixes along with regular security reviews will greatly reduce the attack surface and protect the application from common and critical vulnerabilities.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
HIGH	Debug Mode Enabled	`app configuration (app.config["DEBUG"] = True)`	Running the Flask app in DEBUG mode can expose sensitive information and internal state when exceptions occur, making it	Disable DEBUG mode in production environments by setting app.config['DEBUG'] to False. Use proper error logging mechanisms

Severity	Title	File / Location	Description	Recommendation
			easier for attackers to exploit vulnerabilities.	instead of exposing debug information.
HIGH	Hard-Coded Secret	`API_SECRET variable`	The API_SECRET is hard-coded in the source code. If the source code is exposed, attackers can obtain this secret which might be used to bypass security checks or misuse other services.	Remove hard-coded secrets from the source. Store sensitive data such as API keys in environment variables or secure configuration management systems, and access them securely.
CRITICAL	Command Injection Vulnerability	`run endpoint (/run)`	The 'cmd' parameter from the request is passed directly to os.popen without validation or sanitization, resulting in a command injection vulnerability. Attackers can execute arbitrary system commands.	Avoid using os.popen with untrusted input. Validate and sanitize user inputs or, preferably, use safer alternatives like subprocess.run with a controlled set of commands. Consider implementing a whitelist of allowed commands.
CRITICAL	SQL Injection Vulnerability	`user endpoint (/user)`	The user-supplied 'id' parameter is passed directly to the get_user_raw_query function without proper sanitization. This can lead to SQL injection if the database query is built with string concatenation.	Use parameterized queries or an ORM framework to ensure proper escaping of user inputs. Validate and sanitize the input, and avoid concatenating untrusted data directly into SQL queries.
MEDIUM	Insecure Token Verification	`login endpoint (/login)`	The verify_token_insecure function is used to check the API token without secure practices. If the token verification logic is weak or predictable, it can be exploited by attackers.	Use a robust and well-tested authentication framework for token verification. Ensure that tokens are properly signed and validated using secure algorithms. Consider using established libraries such as PyJWT for handling JWT tokens.
HIGH	Sensitive Configuration Exposure	`config endpoint (/config)`	The endpoint reads and returns the contents of the configuration file (config/settings.json) directly, exposing sensitive configuration details that could contain credentials or other secure settings.	Avoid exposing sensitive configuration files through public endpoints. Implement proper access control and restrict file access only to authorized users. Consider separating public configuration from sensitive data.
HIGH	Weak Token Verification and Hardcoded Secrets	`HARDCODED_TOKENS list and verify_token_insecure function`	The code uses a list of hardcoded tokens and a simple string prefix check for token verification. Any token that starts with 'test-' is accepted, which allows attackers to easily bypass proper authentication. This leads to a critical authentication bypass vulnerability and exposes the system to unauthorized access.	Eliminate hardcoded tokens and avoid using simplistic prefix checks. Instead, implement a robust token verification mechanism using secure, randomly generated tokens, proper token expiry, and storage in a secure database or key management system. Consider leveraging established authentication frameworks and cryptographic libraries to verify token integrity.
HIGH	Hardcoded Sensitive Information	`Configuration File`	The configuration file contains hardcoded database credentials (DB_USER and DB_PASSWORD) and a JWT secret key. Hardcoding sensitive information in source files can lead to security issues, especially if the source code is accidentally exposed via version control or leaks.	Remove hardcoded credentials and secret keys from the source file. Instead, use environment variables or a secure secret management system to store sensitive information. Ensure proper access controls and encryption at rest for these secrets.
HIGH	SQL Injection Vulnerability	`get_user function`	The function concatenates the untrusted 'uid' parameter directly into a raw SQL query without sanitization or parameterization. This could allow an attacker to inject arbitrary SQL commands.	Use parameterized queries or prepared statements. For example, modify the function to use query placeholders: cursor.execute('SELECT * FROM users WHERE id = ?', (uid,)).

Severity	Title	File / Location	Description	Recommendation
HIGH	SQL Injection via String Concatenation	`get_user_raw_query function, SQL query construction line`	The code directly concatenates the 'user_id' parameter into the SQL query string without any sanitization or parameterization, which can lead to SQL injection vulnerabilities if an attacker controls the input.	Use parameterized queries or prepared statements provided by the database API. For example, modify the code to use placeholders (e.g., '?') with the sqlite3 library to safely pass values to the query.
MEDIUM	Hardcoded API Key	`Line defining API_KEY`	The API key is hardcoded in the source code, which can lead to exposure if the code is leaked or inspected. This secret might be used for sensitive operations.	Store sensitive information such as API keys in environment variables or a secure secrets management solution.
CRITICAL	Command Injection	`Function 'run' at route '/run'`	User-provided input from 'cmd' is passed directly to os.popen() without any sanitization or validation, making the code vulnerable to command injection attacks.	Avoid using unsanitized user input in system calls. Validate and sanitize input rigorously. Consider using safer alternatives such as subprocess.run() with a controlled argument list, and if possible, restrict allowed commands.
CRITICAL	SQL Injection	`Function 'user' at route '/user'`	User-provided input from 'id' is concatenated directly into a SQL query string without any sanitization, leading to a SQL injection vulnerability.	Use parameterized queries or prepared statements provided by the database library. Validate and sanitize all user inputs before using them in database queries.
HIGH	XSS Vulnerability: Unescaped User Input in innerHTML	`document.getElementById('welcome').innerHTML assignment`	The code directly inserts untrusted input from the URL query parameter ('name') into the innerHTML of an element without proper sanitization. This can lead to cross-site scripting (XSS) if an attacker provides malicious content (e.g., <script>alert(1)</script>).	Sanitize or escape any untrusted input before inserting it into the DOM. Consider using textContent instead of innerHTML if HTML markup is not required, or use a robust client-side sanitization library.
HIGH	Sensitive Information Logging	`log_login_attempt function, in the logger.warning call`	The function logs user passwords in plain text, exposing sensitive authentication data. Logs are often stored in insecure locations, and if compromised they can be used to gain unauthorized access.	Remove the password from log messages. If needed for debugging, mask or hash the password; ideally, completely avoid logging sensitive information like passwords.

)., "recommendation": "Sanitize or escape any untrusted input before inserting it into the DOM. Consider using textContent instead of innerHTML if HTML markup is not required, or use a robust client-side sanitization library.", "source_file": "C:\\Users\\930140\\AppData\\Local\\Temp\\1\\tmpccp3s10o\\src\\test_project\\static\\app.js" }, { "title": "Sensitive Information Logging", "severity": "high", "location": "log_login_attempt function, in the logger.warning call", "description": "The function logs user passwords in plain text, exposing sensitive authentication data. Logs are often stored in insecure locations, and if compromised they can be used to gain unauthorized access.", "recommendation": "Remove the password from log messages. If needed for debugging, mask or hash the password; ideally, completely avoid logging sensitive information like passwords.", "source_file": "C:\\Users\\930140\\AppData\\Local\\Temp\\1\\tmpccp3s10o\\src\\test_project\\utils\\logger.py" }], "lc_summary": "" }; function downloadJson() { const data = JSON.stringify(reportJson, null, 2); const blob = new Blob([data], { type: "application/json" }); const url = URL.createObjectURL(blob); const a = document.createElement("a"); a.href = url; a.download = "security_report.json"; a.click(); URL.revokeObjectURL(url); }